

Operators and Expressions

Operators (Agarwal & Bansal, 2023)

- Symbols used to compute and compare values and test multiple conditions.

Arithmetic Operators

- All mathematical operations are performed using operators in this category. An operand is the value to be computed.

Name	Operator	Description	Example
Addition	+	Adds an operand	$x = y + z$
Subtraction	-	Subtracts the right operand from the left operand	$x = y - z$
Multiplication	*	Multiplies the operands	$x = y * 3$
Division	/	Divides the left operand by the right operand	$x = y / z$
Modulus	%	Calculates the remainder of an integer division	$x = y \% z$

The + Operator with Numeric Data Types:

- Adding primitive numeric data types results in a primitive numeric data type.
- Since the `byte` data type can hold a range of -128 to +127, Java automatically converts any calculations of numeric data results exceeding this range to the `int` data type (e.g., $50 + 120 = 170$ results in an `int`).

The + Operator with the String Data Type:

- If both operands are strings, the `+` operator joins them to display as a single output.
- If one operand is a string object, the second operand is converted into a string before joining.
- Examples: $7 + 3 = 10$ | $2 + \text{"abc"} = 2\text{abc}$ | $\text{"abc"} + \text{"xyz"} = \text{abcxyz}$

Assignment Operators

- An assignment operator assigns the specific value, variable, and/or functions to another variable.

Name	Operator	Description	Example
Assignment	=	Assigns the value of the right operand to the left operand	\$x = y\$
Add and Assignment	+=	Adds the operands and assigns the result to the left operand	\$x += y\$
Subtract and Assignment	-=	Subtracts the right operand from the left operand and stores the result	\$x -= y\$
Multiple and Assignment	*=	Multiplies the operand by the right operand and stores the result	\$x *= z\$
Divide and Assignment	/=	Divides the left operand by the right operand and stores the result	\$x /= z\$
Modulus and Assignment	%=	Divides the left operand by the right operand and stores the remainder	\$x \% = z\$

- The operation **x operator y** can be represented as **x = x operator y** (e.g., **x += y** is equivalent to **x = x + y**).
- Assigning values to more than one (1) variable at the same time is allowed (e.g., **x = y = 0;**).

Unary Operators

- Operators that require only one operand.

Name	Operator	Description	Example
Unary Plus	+	Represents the positive value	\$+a\$
Unary Minus	-	Represents the negative value	\$-a\$
Increment	++	Increases the value of the operand by 1	\$x++\$
Decrement	--	Decreases the value of the operand by 1	\$x--\$
Logical Complement	!	Inverts the value of a Boolean variable	\$!true\$

- Prefix (Pre-increment/decrement):** Precedes the variable (e.g., `++invar;`). Suppose `$x = 7$`, then the value of `++x` will be `8`.
 - Suffix (Post-increment/decrement):** Follows the variable (e.g., `invar--;`). Suppose `$x = 12$`, then the value of `x--` will be `11`.

Comparison Operators

- All mathematical operations involving comparison between values (often called relational operators).

Name	Operator	Description	Example
Equal to	==	Evaluates whether the operands are equal	\$x == y\$
Not Equal to	!=	Evaluates whether the operands are not equal	\$x != y\$

Greater Than	>	Evaluates whether the left operand is greater than the right operand	\$x > y\$
Less Than	<	Evaluates whether the left operand is less than the right operand	\$x < y\$
Greater Than or Equal to	>=	Evaluates whether the left operand is greater than or equal to the right operand	\$x >= y\$
Less Than or Equal to	<=	Evaluates whether the left operand is less than or equal to the right operand	\$x <= y\$

Shift Operators

- Operators that work on bits of data by moving the bit pattern to the left or right. Can only be used with **integer data types**.

Name	Operator	Description	Example
Right Shift	>>	Bits of the left operand are shifted right by the second operand	\$x = 9 \gg 2\$
Left Shift	<<	Shifts bits to the left, filling zeros at the right	\$x = 9 \ll 2\$
Right Unsigned Shift	>>>	Works like right shift, but 0 is used to replace shifted bits	\$x = -10 \ggg 3\$

- Zeros (0) are used as fillings when shifting in whichever direction.
 - Shifting Positive Numbers Example:** The `int` data type occupies 4 bytes. The rightmost eight bits of 9 are 00001001.
 - Right shift (9 >> 2) results in 2 (00000010).
 - Left shift (9 << 2) results in 36 (00100100).

Bitwise Operators

- Operators that perform operations bit-by-bit. Applied to **byte**, **int**, **short**, **long**, and **char** data types.

Name	Operator	Description	Example
AND	&	Results in a 1 if both bits are 1; any other combination results in 0	$x \& y$
OR		Results in a 0 when both bits are 0; any other combination results in 1	$x \text{ \texttt{ } } y$
XOR	^	Results in a 0 if both bits have the same value, and 1 if different values	$x \wedge y$
Inversion	~	Converts all 1 bits to 0s and all 0 bits to 1s	$\sim x$

- When used with Boolean expressions, if both expressions evaluate to true, the result of using & is true; otherwise, false.

Logical Operators

- Used to combine the results of Boolean expressions. Limited to **Boolean expressions only**.

Name	Operator	Description	Example
Logical AND	&&	Returns true if both operands are non-zero; otherwise false	$x > 4 \&\& y < 8$
Logical OR	,		,

- For Logical AND, the result is true only if both conditions are true. If any condition is false, the result is false.

Conditional Operator

- Used to control the flow of the program using a ternary operator primarily in loop statements.

Name	Operator	Description	Example
Ternary	<code>(condition) ? val1 : val2</code>	Evaluates <code>val1</code> if condition is true and <code>val2</code> if condition is false	<code>\$x = (y > z) ? y : z\$</code>

- `$x$` is assigned the value of `$y$` if it is greater than `$z$`; otherwise, `$x$` is assigned the value of `$z$`.

Order of Precedence of Operators

- Shows the hierarchy wherein an operation with high precedence is performed before others.

Category	Operators
High Precedence	<code>[], ()</code>
Unary	<code>+, -, ~, !, ++, --</code>
Multiplicative	<code>*, /, %</code>
Additive	<code>+, -</code>
Shift	<code><<, >>, >>></code>

Relational	<, <=, >=, >
Equality	=, ==, !=
Bitwise	&, ^,
Logical	&&,
Conditional	?:
Assignment	=, +=, -=, *=, /=, %=

Expressions (Agarwal & Bansal, 2023)

- A combination of variables, constants, literals, and operators to produce a single value.
- An expression statement consists of an expression followed by a semicolon (;) to evaluate an expression and disregard the result.

Types of Expressions:

- **Arithmetic Expression:** Returns a numeric value based on the operators and operands used (e.g., adding \$x\$ and \$y\$ and assigning to \$z\$).
- **Assignment Expression:** Involves assigning a value to a variable (e.g., adding \$4\$ to the value of \$a\$ and assigning it back to \$a\$).
- **Relational Expression:** Compares values using relational operators (e.g., comparing \$x\$ and \$y\$ and assigning the Boolean result).
- **Logical Expression:** Involves logical operators evaluating Boolean conditions (e.g., checking if \$j < k\$ and \$k < 4\$).
- **Conditional Expression:** Uses the ternary operator (?:) to assign a value based on a condition (e.g., assigning \$2\$ or \$3\$ based on whether \$l < m\$).